

Glossaire oral CDA — GreenRoots

Tony Memponteil

Glossaire oral CDA - GreenRoots (Partie 1/2)

1. Gestion de projet et méthode (Agile / Scrum)
2. Architecture et conception
3. Sécurité - la partie la plus scrutée

Glossaire oral CDA - GreenRoots (Partie 2/2)

4. Base de données
5. Frontend
6. Paiement (Stripe)
7. Tests
8. Déploiement et DevOps
9. RGPD et cadre légal
10. Services externes et divers

Comment écouter ce glossaire en podcast sur ton téléphone

Glossaire oral CDA - GreenRoots (Partie 1/2)

Comment utiliser ce document. Pour chaque terme : une définition simple, son rôle concret dans GreenRoots, et une ligne « Si le jury creuse » avec la question piège probable et la réponse. Le texte est écrit pour être lisible à voix haute (podcast). La partie 2 couvre les données, le front, le paiement, les tests, le déploiement et le RGPD.

Règle d'or à l'oral. Toujours répondre en trois temps : (1) ce que c'est, (2) pourquoi je l'ai choisi dans GreenRoots, (3) la limite ou l'alternative que j'ai écartée. Un jury valide une compétence quand tu montres que tu as *décidé*, pas juste utilisé.

1. Gestion de projet et méthode (Agile / Scrum)

Agile. Une approche de développement itérative et incrémentale : on livre par petits morceaux fonctionnels plutôt que tout à la fin. L'opposé, c'est le cycle en V (ou « cascade »), où l'on spécifie tout, puis on développe tout, puis on teste tout. Dans GreenRoots on a travaillé en Agile, avec la méthode Scrum. Si le jury creuse : « pourquoi Agile plutôt que le cycle en V ? » Réponse : parce que les besoins évoluent, l'itératif permet de corriger le tir à chaque sprint et de livrer de la valeur tôt ; le cycle en V convient mieux à un projet aux besoins figés, comme l'aéronautique.

Scrum. Un cadre concret pour appliquer l'Agile, avec des rôles, des événements (cérémonies) et des artefacts. Les rôles : le Product Owner qui porte le besoin et priorise, le Scrum Master qui fluidifie et enlève les obstacles, et l'équipe de développement. Dans GreenRoots : Marie était PO, Léo Scrum Master, et Vincent, Naomi et moi les développeurs. Si le jury creuse : « quelle est la différence entre le PO et le Scrum Master ? » Réponse : le PO décide *quoi* faire et dans quel ordre (la valeur métier) ; le Scrum Master veille au *comment* de l'équipe (le processus), il n'est pas un chef de projet.

Sprint. Une itération de durée fixe (chez nous, une semaine) au bout de laquelle on livre un incrément fonctionnel. Nos quatre sprints racontent une histoire : sprint 0 la conception (cahier des charges, maquettes, architecture, base de données), sprint 1 les fondations techniques, sprint 2 le MVP e-commerce complet, sprint 3 la qualité et la mise en production. Si le jury creuse : « qu'est-ce qu'un sprint 0 ? » Réponse : un sprint préparatoire, avant de coder, pour poser les bases (conception, environnement) ; il n'y a pas d'incrément livrable mais des livrables de conception.

Backlog. La liste priorisée de tout ce qui reste à faire, exprimée surtout en user stories. Le PO l'ordonne par valeur. On le gère dans Trello, en Kanban. Si le jury creuse : « backlog produit ou backlog de sprint ? » Réponse : le backlog produit est la liste globale ; le backlog de sprint est le sous-ensemble qu'on s'engage à faire pendant le sprint courant.

User story. Une expression du besoin du point de vue de l'utilisateur, au format « En tant que [rôle], je veux [action] afin de [bénéfice] ». Ça force à parler valeur, pas solution technique. GreenRoots en compte vingt-huit, réparties entre le visiteur, l'utilisateur connecté et l'administrateur, plus une transverse. Si le jury creuse : « différence entre une user story et un cas d'utilisation ? » Réponse : la user story est un besoin court côté produit ; le cas d'utilisation (use case UML) formalise l'interaction entre un acteur et le système, avec les relations include et extend. On regroupe plusieurs user stories dans un cas d'utilisation.

Kanban. Un tableau visuel avec des colonnes (à faire, en cours, terminé) où l'on déplace les tâches. C'est ce que Trello matérialise. Si le jury creuse : « Scrum ou Kanban ? » Réponse : on a fait du Scrum (sprints à durée fixe, cérémonies) en visualisant le travail sur un tableau Kanban ; les deux se combinent, on appelle ça Scrumban.

MVP (Minimum Viable Product). La plus petite version qui apporte déjà de la valeur et qu'on peut mettre entre les mains d'un utilisateur. Notre MVP : catalogue, authentification, tunnel d'achat complet avec vrai paiement Stripe, espace utilisateur, back-office admin. Ce qui est hors périmètre (carte interactive, parrainage, multilingue) est écarté *volontairement*, pas oublié. Si le jury creuse : « pourquoi Stripe est dans le MVP et pas la carte de plantation ? » Réponse : le paiement est le cœur d'un e-commerce, sans lui il n'y a pas de produit viable ; la carte est un plus d'engagement, reportable.

Pull Request et code review. Une pull request (PR) est une demande d'intégration d'une branche dans une autre ; elle est relue par un pair (code review) avant d'être fusionnée. Chez nous, chaque fonctionnalité passait par une PR relue, et la CI devait être verte. Si le jury creuse : « pourquoi imposer la revue ? » Réponse : elle attrape des bugs tôt, diffuse la connaissance dans l'équipe et maintient une cohérence de style ; c'est une procédure qualité.

Conventional Commits. Une convention de nommage des messages de commit (par exemple « feat: », « fix: », « docs: ») qui rend l'historique lisible et automatisable. Si le jury creuse : « à quoi ça sert concrètement ? » Réponse : à générer des changelogs automatiquement et à comprendre l'historique d'un coup d'œil ; c'est de la rigueur de collaboration.

Git flow. Une organisation des branches Git : une branche de production (main), une branche d'intégration (develop), et des branches de fonctionnalité. Nos workflows CI/CD sont calqués dessus. Si le jury creuse : « pourquoi une branche develop ? » Réponse : pour intégrer et tester l'ensemble avant de promouvoir en production ; main reste toujours déployable.

2. Architecture et conception

Architecture trois tiers (ou trois couches). Une séparation en trois couches déployables indépendamment : la présentation (le front), la logique métier (l'API), et les données (la base). Dans GreenRoots : une SPA React, une API REST NestJS, et PostgreSQL

plus Redis. Si le jury creuse : « trois tiers ou trois couches, c'est pareil ? » Réponse : on parle de « tiers » quand les couches sont sur des machines/process séparés (réparti), et de « couches » pour la séparation logique ; ici c'est bien réparti, donc trois tiers.

SPA (Single Page Application). Une application web qui se charge une fois et met à jour la page dynamiquement en JavaScript, sans recharger, en appelant une API. React est notre framework de SPA. Si le jury creuse : « quelle limite de la SPA ? » Réponse : le référencement (SEO) et le premier affichage sont moins bons qu'en rendu serveur, parce que le contenu arrive après le chargement du JavaScript ; pour un site vitrine on choisirait plutôt du rendu serveur (SSR), mais pour une application derrière authentification la SPA est adaptée.

API REST. Une interface entre le front et le back qui suit le style REST : des ressources adressées par des URL, manipulées avec les verbes HTTP (GET pour lire, POST pour créer, PATCH pour modifier, DELETE pour supprimer), sans état côté serveur entre deux requêtes. Si le jury creuse : « REST est-il sans état, alors que vous avez de l'authentification ? » Réponse : oui, chaque requête porte elle-même son jeton d'authentification (dans un cookie), le serveur ne garde pas de session en mémoire ; c'est ce qui rend l'API facilement scalable.

Découplage front / back. Le front et le back sont deux projets séparés qui communiquent par l'API. On n'a pas utilisé un framework tout-en-un comme Next.js. Si le jury creuse : « pourquoi ne pas avoir pris Next.js ? » Réponse : le découplage permet de faire évoluer ou remplacer chaque partie indépendamment, et de réutiliser l'API pour une future application mobile ; Next.js aurait mélangé rendu et API, on l'a écarté volontairement.

NestJS. Un framework backend Node.js structuré, en TypeScript, qui impose une architecture modulaire et l'injection de dépendances. Il organise le code en modules par domaine (auth, orders, checkout, etc.), chacun avec ses contrôleurs, services et DTO. Si le jury creuse : « pourquoi NestJS plutôt qu'Express nu ? » Réponse : Express est minimaliste et laisse tout organiser soi-même ; NestJS apporte une structure, l'injection de dépendances et des garde-fous, ce qui aide sur un projet à plusieurs et sur la durée.

Injection de dépendances. Un composant ne crée pas lui-même les objets dont il dépend, il les reçoit de l'extérieur (le framework les fournit). Ça découple le code et le rend testable, parce qu'on peut injecter une fausse dépendance dans les tests. Si le jury creuse : « donnez un exemple concret. » Réponse : mon service reçoit Prisma par injection ; dans un test unitaire je lui injecte un faux Prisma (un mock) pour tester la logique sans vraie base de données.

Contrôleur, service, couche d'accès aux données. Le contrôleur reçoit la requête HTTP et la valide ; le service contient les règles métier ; la couche d'accès (Prisma) parle à la base. C'est le trajet d'une requête dans le back. Si le jury creuse : « pourquoi ne pas mettre la logique dans le contrôleur ? » Réponse : pour séparer les responsabilités ; le contrôleur ne s'occupe que du transport HTTP, le service reste testable et réutilisable indépendamment du web.

Architecture modulaire en couches, pas hexagonale. Il faut être honnête : le code est organisé en couches (contrôleur, service, Prisma), mais ce n'est pas une architecture hexagonale stricte, parce que le service dépend directement de Prisma. Si le jury creuse : « c'est de l'hexagonal ? » Réponse : non, et c'est un choix pragmatique ; l'hexagonal isolerait la base derrière une interface (un port) pour pouvoir en changer, mais pour ce projet le gain ne justifiait pas la complexité. Assume-le, ne prétends pas le contraire.

Atomic Design. Une méthode d'organisation des composants front par niveau de complexité : les atomes (un bouton), les molécules (un champ avec son label), les organismes (un formulaire complet). Attention : dans GreenRoots on a seulement les trois niveaux atomes, molécules, organismes - pas de templates ni pages (les pages sont dans le dossier des routes). Si le jury creuse : « avez-vous les cinq niveaux ? » Réponse : non, trois, plus un découpage par fonctionnalité (feature-based) ; j'ai adapté la méthode à la taille du projet.

Éco-conception. Concevoir en limitant la consommation de ressources. Dans GreenRoots : vite produit un build léger, Tailwind purge le CSS inutilisé, shadcn/ui n'importe que les composants utilisés, et le cache Redis évite des requêtes SQL redondantes. Si le jury creuse : « donnez une mesure. » Réponse : je n'ai pas de bilan carbone chiffré, mais des leviers concrets : moins de JavaScript envoyé, moins de requêtes en base ; c'est un axe identifié, mesurable avec Lighthouse côté performance.

Reverse proxy (Nginx). Un serveur placé devant l'application qui reçoit toutes les requêtes et les redirige : il sert les fichiers du front et route les appels commençant par /api vers le back. Le navigateur ne parle donc jamais directement au back ni à la base. Si le jury creuse : « pourquoi un reverse proxy ? » Réponse : un seul point d'entrée, la gestion du HTTPS centralisée, la compression, le cache des fichiers, et une surface d'attaque réduite (le back n'est pas exposé publiquement).

3. Sécurité - la partie la plus scrutée

3.1 Les référentiels

OWASP. L'Open Worldwide Application Security Project, une organisation à but non lucratif qui publie des ressources de sécurité applicative, dont le célèbre « Top 10 » des risques les plus critiques des applications web. Si le jury creuse : « OWASP, c'est une norme obligatoire ? » Réponse : non, c'est un référentiel de bonnes pratiques reconnu mondialement, pas une norme légale ; je m'en suis servi comme grille pour auditer mon code.

OWASP Top 10. Un classement des dix catégories de failles les plus répandues, mis à jour tous les quelques années. Important : la numérotation change entre les éditions. Dans mon dossier j'ai utilisé l'édition 2025, où l'on retrouve notamment A01 Broken Access Control, A02 Security Misconfiguration, A04 Cryptographic Failures et A05 Injection (qui englobe le

XSS). Si un juré cite un autre numéro, c'est probablement l'édition 2021 (où l'injection était A03). Si le jury creuse : « pourquoi le XSS est dans Injection ? » Réponse : parce qu'une injection, c'est faire exécuter au système des données interprétées comme du code ; le XSS injecte du script dans une page, c'est conceptuellement la même famille. Conseil : parle d'abord du *nom* de la faille, le numéro est secondaire.

ANSSI. L'Agence Nationale de la Sécurité des Systèmes d'Information, l'autorité française en cybersécurité. Elle publie des guides, dont un guide de sécurisation des sites web dont je me suis inspiré. Si le jury creuse : « qu'avez-vous repris de l'ANSSI ? » Réponse : les recommandations de base - HTTPS partout, en-têtes de sécurité, mots de passe hachés et salés, principe du moindre privilège, validation des entrées ; c'est cohérent avec OWASP.

EBIOS. Une méthode française d'analyse des risques (Expression des Besoins et Identification des Objectifs de Sécurité), portée par l'ANSSI. C'est de là que vient la grille DICP. Si le jury creuse : « avez-vous fait une analyse EBIOS complète ? » Réponse : non, une analyse EBIOS Risk Manager formelle serait disproportionnée ici ; j'ai repris son vocabulaire (les critères DICP) pour structurer ma réflexion sécurité.

DICP. Les quatre critères de sécurité d'une information : Disponibilité (elle est accessible quand on en a besoin), Intégrité (elle n'est pas altérée), Confidentialité (seuls les autorisés y accèdent), et Preuve, aussi appelée traçabilité (on peut prouver qui a fait quoi). Dans GreenRoots, un exemple par lettre : Disponibilité, le cron de réconciliation et les healthchecks ; Intégrité, les transactions atomiques et les contraintes en base ; Confidentialité, le jeton en cookie HttpOnly, bcrypt, le rôle relu en base ; Preuve, les factures numérotées et l'anonymisation qui conserve l'historique. Si le jury creuse : « quelle différence entre DICP et le triptyque CIA ? » Réponse : le monde anglophone parle de CIA (Confidentiality, Integrity, Availability), la France y ajoute la Preuve, d'où DICP ; c'est plus complet pour la traçabilité.

Sécurité « by design ». Penser la sécurité dès la conception, pas la rajouter à la fin. La grille DICP illustre cette démarche. Si le jury creuse : « donnez un exemple de décision by design. » Réponse : dès la conception j'ai décidé de stocker le jeton d'authentification dans un cookie HttpOnly plutôt que dans le localStorage, précisément pour qu'une faille XSS ne puisse pas le voler ; c'était une décision d'architecture, pas un correctif.

Défense en profondeur. Empiler plusieurs couches de protection pour qu'aucune faille unique ne soit fatale : si une couche cède, une autre protège. Dans GreenRoots la validation des entrées est faite côté client avec Zod, revalidée côté API avec les DTO, et l'accès aux données passe par des requêtes paramétrées. Si le jury creuse : « la validation côté client ne suffit-elle pas ? » Réponse : non, jamais ; le client est manipulable (on peut appeler l'API directement) ; la validation client est du confort d'expérience, la vraie barrière est côté serveur.

3.2 Authentification et autorisation

Authentification vs autorisation. L'authentification répond à « qui es-tu ? » (prouver son identité au login) ; l'autorisation répond à « qu'as-tu le droit de faire ? » (les permissions). Si le jury creuse : « où gérez-vous chacune ? » Réponse : l'authentification à la connexion (stratégie Local de Passport, vérification du mot de passe) ; l'autorisation à chaque requête via les guards (JwtAuthGuard puis RolesGuard).

JWT (JSON Web Token). Un jeton signé qui contient des informations (par exemple l'identifiant de l'utilisateur) et une signature qui garantit qu'il n'a pas été modifié. Le serveur peut le vérifier sans stocker de session. Si le jury creuse : « le contenu d'un JWT est-il chiffré ? » Réponse : non, il est seulement signé, donc lisible par tous (encodé en base64) ; il ne faut donc jamais y mettre de secret ; sa sécurité vient de la signature qui empêche la falsification. Deuxième piège classique : « comment révoquez-vous un JWT, puisqu'il est autoportant ? » Réponse : par une liste noire dans Redis au logout (voir plus bas).

Structure d'un JWT (à savoir réciter). Un JWT a trois parties séparées par des points : header, payload, signature, chacune encodée en base64url. Le header indique l'algorithme de signature (souvent HS256). Le payload porte les informations : l'identifiant de l'utilisateur (le champ sub), éventuellement le rôle, la date d'émission (iat) et la date d'expiration (exp, vingt-quatre heures chez moi). La signature est un HMAC-SHA256 calculé sur le header et le payload encodés, avec un secret que seul le serveur connaît (JWT_SECRET). À chaque requête, le serveur recalcule cette signature ; si elle correspond, le token est authentique et n'a pas été modifié. Le payload n'est pas chiffré, seulement signé : il est donc lisible par tous, on n'y met jamais de secret ; toute la sécurité vient de l'impossibilité de forger une signature valide sans connaître le secret.

Cookie HttpOnly, Secure, SameSite. Le jeton JWT est stocké dans un cookie avec trois attributs. HttpOnly : le cookie est inaccessible au JavaScript, donc une faille XSS ne peut pas le voler. Secure : le cookie n'est envoyé qu'en HTTPS. SameSite=Lax : le navigateur n'envoie pas le cookie lors de requêtes venant d'un autre site, ce qui protège du CSRF. Nuance honnête à connaître : dans mon code, Secure n'est activé qu'en production (en HTTP local de développement, il serait bloquant). Si le jury creuse : « pourquoi le cookie plutôt que le localStorage ? » Réponse : le localStorage est lisible en JavaScript, donc vulnérable au vol de jeton par XSS ; le cookie HttpOnly ferme cette porte. C'est ma décision de sécurité la plus importante côté front (une migration depuis le localStorage).

bcrypt. Un algorithme de hachage spécialement conçu pour les mots de passe : lent volontairement, et intégrant un « sel ». Dans GreenRoots, dix tours (rounds). Si le jury creuse : « différence entre hachage et chiffrement ? » Réponse : le chiffrement est réversible avec une clé ; le hachage est à sens unique, on ne peut pas retrouver le mot de passe d'origine, on ne fait que recomparer. Deuxième piège : « c'est quoi le sel ? » Réponse : une valeur aléatoire unique ajoutée à chaque mot de passe avant hachage, pour que deux utilisateurs avec le même mot de passe aient des empreintes différentes, ce qui casse les

attaques par tables précalculées (rainbow tables) ; bcrypt gère le sel automatiquement.

Troisième piège : « pourquoi lent, c'est un défaut ? » Réponse : non, c'est voulu, la lenteur ralentit les attaques par force brute ; le nombre de tours règle ce coût.

SHA-256. Une fonction de hachage rapide et générale. Dans GreenRoots je l'utilise pour hacher les jetons de réinitialisation de mot de passe et de vérification d'email avant de les stocker en base. Piège important à ne pas confondre : je n'utilise pas SHA-256 pour les mots de passe (c'est bcrypt) ni pour la liste noire des JWT (là c'est une simple empreinte). Si le jury creuse : « pourquoi SHA-256 pour ces jetons et bcrypt pour les mots de passe ? »

Réponse : ces jetons sont déjà aléatoires et à durée de vie courte, un hachage rapide suffit à rendre une fuite de base inexploitable ; les mots de passe, eux, sont choisis par les humains (souvent faibles), donc il faut un hachage lent et salé comme bcrypt.

Passport. Une bibliothèque d'authentification pour Node, qui fonctionne par « stratégies ». Dans GreenRoots deux stratégies : Local (vérifie identifiant et mot de passe au login) et JWT (protège les routes en vérifiant le jeton). Si le jury creuse : « c'est quel patron de conception ? » Réponse : le patron Strategy - on encapsule chaque méthode d'authentification derrière une interface commune, interchangeable.

Guard. Dans NestJS, un composant qui décide si une requête a le droit de passer, avant d'atteindre le contrôleur. JwtAuthGuard vérifie que l'utilisateur est authentifié ; RolesGuard vérifie qu'il a le bon rôle. Si le jury creuse : « où placez-vous les guards ? » Réponse : de façon déclarative sur les routes, avec des décorateurs comme @Roles ; c'est lisible et centralisé.

RBAC (contrôle d'accès basé sur les rôles). On attribue des permissions à des rôles, et des rôles aux utilisateurs. Attention à la nuance : en base, GreenRoots a deux rôles réels, « user » et « admin ». Le « visiteur » n'est pas un rôle en base, c'est simplement un utilisateur non authentifié. Sur le diagramme de cas d'utilisation je parle donc de trois *acteurs* (visiteur, utilisateur, admin), pas de trois rôles. Si le jury creuse : « combien de rôles avez-vous ? » Réponse : deux rôles techniques en base, plus l'acteur visiteur non connecté ; la hiérarchie d'accès est conceptuelle, gérée par les guards, ce n'est pas de l'héritage objet.

Le rôle relu en base à chaque requête. Point fort à mettre en avant : ma stratégie JWT ne fait pas confiance au rôle inscrit dans le jeton, elle recharge l'utilisateur depuis la base à chaque requête et lit son rôle réel. Si le jury creuse : « pourquoi ne pas prendre le rôle dans le jeton, ce serait plus rapide ? » Réponse : parce que si on rétrograde un admin, un jeton encore valide continuerait à donner les droits admin ; en relisant la base, un changement de rôle est pris en compte immédiatement.

Anti-force-brute (brute-force). Une protection contre les tentatives massives de mots de passe. Dans GreenRoots, un compteur d'échecs par email dans Redis : au-delà de cinq échecs, le compte est bloqué quinze minutes. Si le jury creuse : « et si Redis tombe ? » Réponse : la protection s'ouvre (fail-open), c'est-à-dire qu'on laisse passer plutôt que de bloquer tout le monde ; c'est un choix assumé de disponibilité, complété par un rate-limit global.

Rate limiting / Throttling. Limiter le nombre de requêtes par unité de temps pour éviter les abus. NestJS le fournit via un ThrottlerGuard. Point important : le webhook Stripe est explicitement exclu du rate-limit (annotation SkipThrottle), parce que Stripe rejoue ses notifications et ne doit jamais être bridé. Si le jury creuse : « différence entre le throttling global et l'anti-force-brute ? » Réponse : le throttling limite tout le trafic par adresse ; l'anti-force-brute cible spécifiquement les échecs de login par compte.

Liste noire de JWT (blacklist). Comme un JWT est autoportant, on ne peut pas l'« effacer » côté serveur. À la déconnexion, on inscrit donc le jeton dans une liste noire Redis, avec une durée de vie égale au temps restant du jeton ; la stratégie JWT vérifie cette liste à chaque requête. Nuance honnête : la clé de la liste n'est pas un hachage cryptographique, c'est une empreinte simple du jeton. Si le jury creuse : « ça ne casse pas le côté sans état du JWT ? » Réponse : partiellement, oui, c'est un compromis assumé pour permettre la révocation immédiate ; Redis est en mémoire donc la vérification reste très rapide. Alternative plus standard (à citer en perspectives) : des access tokens courts (par exemple quinze minutes) plus un refresh token ; comme l'access token expire vite, la déconnexion n'a plus besoin de blacklist et on supprime le lookup Redis à chaque requête. Plus robuste, mais plus complexe (endpoint de rafraîchissement, rotation, stockage du refresh) ; pour ce MVP j'ai préféré vingt-quatre heures plus blacklist.

HMAC. Un code d'authentification de message basé sur un secret partagé : il prouve à la fois l'intégrité d'un message et son origine. Stripe signe ses webhooks en HMAC ; mon serveur vérifie cette signature avec le secret partagé pour être sûr que la notification vient bien de Stripe. Si le jury creuse : « différence entre HMAC et une simple signature ? » Réponse : HMAC repose sur un secret symétrique partagé (Stripe et moi le connaissons) ; une signature asymétrique utiliserait une paire clé publique / clé privée ; ici Stripe impose le HMAC.

3.3 Les failles et leurs parades

Broken Access Control (contrôle d'accès défaillant). La faille numéro un du Top 10 : un utilisateur accède à une ressource qui ne lui appartient pas. Deux niveaux de parade dans GreenRoots : par rôle (les guards user/admin) et par propriétaire (on vérifie que la commande appartient bien à l'utilisateur, sinon on renvoie une erreur 403 Forbidden). Si le jury creuse : « donnez un exemple précis. » Réponse : dans confirmCheckout, je compare l'identifiant du propriétaire de la commande à celui de l'utilisateur connecté ; s'ils diffèrent, 403, impossible de confirmer la commande d'un autre.

403 Forbidden. Le code HTTP qui signifie « tu es authentifié mais tu n'as pas le droit ». À distinguer du 401 Unauthorized, qui veut dire « tu n'es pas authentifié ». Si le jury creuse : « 401 ou 403 ? » Réponse : 401 quand l'identité manque ou est invalide (pas de jeton) ; 403 quand l'identité est connue mais les droits manquent.

Injection SQL. Faire exécuter du code SQL malveillant en glissant des instructions dans un champ de saisie. La parade : les requêtes paramétrées, où les données sont envoyées séparément de la requête et ne sont jamais interprétées comme du code. Prisma génère automatiquement des requêtes paramétrées, donc l'injection SQL est neutralisée par construction. Si le jury creuse : « et si vous écrivez du SQL brut ? » Réponse : Prisma permet le queryRaw, et là il faut absolument passer par des paramètres, jamais par de la concaténation de chaînes ; dans GreenRoots je n'en ai pas besoin.

XSS (Cross-Site Scripting). Injecter du JavaScript malveillant dans une page pour qu'il s'exécute chez d'autres utilisateurs (par exemple pour voler des données). Parades dans GreenRoots : React échappe le HTML par défaut (je n'utilise jamais dangerouslySetInnerHTML), Helmet ajoute une politique de sécurité de contenu, et surtout le cookie HttpOnly rend le jeton invisible au JavaScript même en cas de XSS. Si le jury creuse : « quels types de XSS connaissez-vous ? » Réponse : le XSS stocké (le script est enregistré en base et resservi), le XSS réfléchi (renvoyé immédiatement dans la réponse), et le XSS basé sur le DOM (côté client) ; React protège des trois tant qu'on n'injecte pas de HTML brut.

CSRF (Cross-Site Request Forgery). Faire exécuter à ton insu une action sur un site où tu es connecté, depuis un autre site. Parades : le cookie SameSite=Lax (le navigateur n'envoie pas le cookie en contexte cross-site) et le CORS restrictif (une seule origine autorisée). Si le jury creuse : « pourquoi pas de jeton anti-CSRF classique ? » Réponse : avec un cookie SameSite=Lax et une API consommée uniquement par mon front sur une origine unique, le vecteur CSRF est déjà fermé ; un jeton anti-CSRF serait une ceinture en plus des bretelles, envisageable en durcissement.

CORS (Cross-Origin Resource Sharing) - le point qui prête à confusion.

Commençons par la règle de base du navigateur, la « politique de même origine » (same-origin policy) : par défaut, un JavaScript chargé depuis un site A n'a pas le droit de lire la réponse d'une API sur un autre domaine B. C'est le navigateur qui applique cette interdiction, tout seul, pour protéger l'utilisateur. CORS est le mécanisme qui permet au serveur B de lever cette interdiction pour des origines précises : mon backend renvoie des en-têtes de réponse (Access-Control-Allow-Origin, et compagnie) qui disent en substance « j'autorise le site de mon front à lire mes réponses ». Le navigateur lit ces en-têtes et décide alors de laisser, ou non, le JavaScript accéder à la réponse.

Voici la phrase qui résout ta confusion : le serveur déclare la règle, le navigateur l'applique. On configure CORS côté backend (chez moi, l'origine du front dans la variable FRONTEND_URL) parce que seul le serveur sait quelles origines sont légitimes ; mais c'est le navigateur qui fait respecter la règle, d'où l'impression que « c'est le navigateur qui dit oui ». Le serveur ne peut rien imposer au navigateur, il ne fait qu'envoyer une consigne. Image mentale : le serveur est le videur qui écrit la liste des invités (les origines autorisées), le navigateur est le portier qui vérifie cette liste à l'entrée. Le serveur écrit la liste (configuration backend), le navigateur contrôle (application). Détail : pour les requêtes non triviales, le

navigateur envoie même d'abord une requête de vérification (méthode OPTIONS, le « preflight ») pour demander au serveur « telle origine a-t-elle le droit ? » avant d'envoyer la vraie requête.

Si le jury creuse : « le CORS protège-t-il l'API ? » Réponse : non, il protège l'utilisateur dans son navigateur - il empêche un site malveillant de lire mes données via le navigateur de la victime. Un appel direct hors navigateur (curl, un autre serveur) ignore complètement le CORS. Donc le CORS complète, mais ne remplace jamais, l'authentification et les guards côté serveur. Piège de mon dossier : le récit « avant toutes origines, après une seule » est une évolution ; dans le code actuel on ne voit que l'état final restreint, présente-le comme un historique.

Cookie SameSite=Lax - clarification. Exactement le même principe que CORS : le serveur pose l'attribut (dans l'en-tête Set-Cookie), le navigateur l'applique. SameSite ne veut surtout pas dire « ce cookie n'est valide que s'il vient de tel domaine » - le serveur ne valide rien à la réception. SameSite dit au navigateur quand joindre, ou non, le cookie à une requête, selon le contexte de cette requête. Deux contextes : « same-site » (la requête part de mon propre site) et « cross-site » (elle part d'un autre site). En mode Lax : le navigateur envoie le cookie pour les requêtes issues de mon site, et pour une navigation de premier niveau depuis un autre site (l'utilisateur clique un lien qui l'amène chez moi) ; mais il ne l'envoie pas pour une sous-requête cross-site déclenchée en arrière-plan par le JavaScript d'un autre site (un POST, un fetch, une image, une iframe). Résultat concret : si un site malveillant tente, à l'insu de l'utilisateur connecté, d'envoyer une requête à mon API, le navigateur n'attache pas le cookie d'authentification, donc la requête forgée échoue - c'est la parade contre le CSRF. Précision de vocabulaire qu'un juré pointilleux peut demander : CORS raisonne en « origine » (protocole + domaine + port), SameSite raisonne en « site » (le domaine enregistrable, par exemple mon-domaine.fr, sous-domaines confondus). Dans les deux cas, retiens le fil commun : c'est le serveur qui déclare, et le navigateur qui applique.

Helmet. Un middleware qui ajoute des en-têtes HTTP de sécurité. Les principaux : Content-Security-Policy (limite les sources de scripts, contre le XSS), HSTS (force le HTTPS), X-Frame-Options (empêche l'affichage du site dans une iframe, contre le clickjacking). Nuance honnête : HSTS n'est activé qu'en production dans mon code. Si le jury creuse : « c'est quoi le clickjacking ? » Réponse : piéger un utilisateur en superposant mon site invisible dans une iframe sur un site malveillant pour lui faire cliquer sans le savoir ; X-Frame-Options l'empêche.

Security Misconfiguration (mauvaise configuration). La catégorie qui regroupe les réglages de sécurité oubliés ou trop permissifs : en-têtes absents, ports exposés, comptes par défaut. Ma parade emblématique : ajouter Helmet là où il n'y avait aucun en-tête de sécurité. Si le jury creuse : « d'autres exemples chez vous ? » Réponse : en production, le back n'est pas exposé publiquement (expose et non ports), les conteneurs tournent en utilisateur non-root, et les variables d'environnement sont validées au démarrage par Joi.

Cryptographic Failures (défaillances cryptographiques). La catégorie des données sensibles mal protégées : mots de passe en clair, absence de HTTPS, algorithmes faibles. Ma parade : les jetons de réinitialisation, autrefois stockés en clair, sont désormais hachés en SHA-256. Si le jury creuse : « les données bancaires ? » Réponse : je n'en stocke aucune ; c'est Stripe qui les gère, je ne reçois qu'un identifiant de paiement, donc le risque cryptographique sur ce point est externalisé.

ValidationPipe, whitelist, forbidNonWhitelisted. Dans NestJS, la ValidationPipe valide et nettoie les données entrantes selon les DTO. L'option whitelist supprime tout champ non déclaré ; forbidNonWhitelisted rejette carrément la requête si elle contient un champ inattendu. Si le jury creuse : « à quoi ça sert contre les attaques ? » Réponse : à empêcher qu'un attaquant injecte des champs non prévus (par exemple un champ « role » pour s'auto-promouvoir admin) ; c'est une barrière contre l'élévation de privilèges (mass assignment).

DTO (Data Transfer Object) et class-validator. Un DTO est un objet qui décrit la forme attendue des données à l'entrée de l'API ; class-validator ajoute des règles de validation par annotations. C'est la validation côté serveur, au bord de l'API, en complément de Zod côté client. Si le jury creuse : « pourquoi valider deux fois, client et serveur ? » Réponse : défense en profondeur ; le client, c'est l'ergonomie (feedback immédiat) ; le serveur, c'est la sécurité (la seule barrière qui compte vraiment).

Glossaire oral CDA - GreenRoots (Partie 2/2)

Suite de la partie 1 (gestion de projet, architecture, sécurité). Ici : base de données, frontend, paiement, tests, déploiement, RGPD, services. La dernière section explique comment écouter ces notes en podcast sur ton téléphone.

4. Base de données

4.1 Modélisation MERISE

MERISE. Une méthode française de conception de bases de données, en trois niveaux d'abstraction : le modèle conceptuel (MCD), le modèle logique (MLD), le modèle physique (MPD). On part du métier et on descend vers le technique. Si le jury creuse : « pourquoi trois niveaux ? » Réponse : pour séparer le quoi du comment ; le conceptuel décrit le métier sans se soucier de la technique, le physique dépend du moteur de base choisi ; ça rend la conception plus claire et plus portable.

MCD (Modèle Conceptuel de Données). La vue métier : les entités (par exemple Utilisateur, Commande, Arbre), leurs propriétés, et les associations entre elles avec des cardinalités. Indépendant de tout moteur de base. Si le jury creuse : « c'est quoi une cardinalité ? » Réponse : elle indique combien d'occurrences d'une entité participent à une association, par exemple « un utilisateur passe zéro à plusieurs commandes, une commande appartient à un et un seul utilisateur ». On la note souvent 0,n et 1,1.

MLD (Modèle Logique de Données). La traduction du MCD en tables relationnelles, en appliquant les règles de passage : chaque entité devient une table, les associations deviennent des clés étrangères ou des tables de liaison. Si le jury creuse : « comment passe-t-on une association plusieurs-à-plusieurs ? » Réponse : elle devient une table d'association portant les deux clés étrangères ; par exemple, une commande contient plusieurs arbres et un arbre est dans plusieurs commandes, ce qui donne la table OrderItem avec une contrainte d'unicité sur le couple commande-arbre.

MPD (Modèle Physique de Données). Le schéma réel implanté dans le moteur, ici PostgreSQL : les types exacts (entier, texte), les enums, les contraintes, les index. Chez moi il est matérialisé par le schéma Prisma. Piège à connaître : mon schéma n'a pas d'index explicites (@@index), seulement des contraintes d'unicité (@@unique). Si le jury creuse : « avez-vous des index pour la performance ? » Réponse honnête : les clés primaires et les contraintes uniques créent des index automatiquement ; je n'ai pas ajouté d'index supplémentaires, ce serait un axe d'optimisation si le volume augmentait. Ne prétends pas avoir des @@index, tu n'en as pas.

Clé primaire, clé étrangère, contrainte d'unicité. La clé primaire identifie de façon unique une ligne. La clé étrangère référence la clé primaire d'une autre table, garantissant l'intégrité référentielle. La contrainte d'unicité empêche les doublons sur une ou plusieurs colonnes. Si le jury creuse : « donnez un exemple d'unicité métier chez vous. » Réponse : sur OrderItem, l'unicité du couple commande-arbre évite d'avoir deux lignes pour le même arbre dans une commande (on incrémente la quantité à la place) ; sur OrderAddress, l'unicité du couple commande-type évite deux adresses de facturation sur une même commande.

Enum. Un type énuméré : une colonne ne peut prendre qu'une valeur parmi une liste fixe. Dans GreenRoots : le statut de commande (PENDING, CONFIRMED, PLANTED, CANCELLED), le statut de paiement (PENDING, PROCESSING, PAID, FAILED, REFUNDED), le type d'adresse (facturation, livraison). Si le jury creuse : « pourquoi un enum plutôt qu'un texte libre ? » Réponse : pour l'intégrité (impossible d'avoir un statut invalide) et la lisibilité ; ces enums sont le support des machines à états des commandes.

Le prix stocké en centimes (type entier). Les montants sont stockés en entiers, en centimes, pas en nombres à virgule. Si le jury creuse : « pourquoi pas un type décimal ? » Réponse : pour éviter les erreurs d'arrondi des flottants (0,1 + 0,2 ne fait pas exactement 0,3 en binaire) ; c'est aussi le format qu'attend Stripe, qui raisonne en plus petite unité monétaire.

PostGIS. Une extension de PostgreSQL qui ajoute des types et fonctions géographiques (coordonnées, distances, zones). Dans GreenRoots, la localisation des plantations utilise un type geometry. Si le jury creuse : « vous l'exploitez vraiment ? » Réponse honnête : la colonne géographique existe et prépare la future carte interactive de plantation ; l'exploitation cartographique complète est hors périmètre du MVP, mais le socle est posé.

Séparation Tree et TreeStock. L'arbre (son nom, son prix, sa description) et son stock sont dans deux tables séparées, reliées un-à-un. Si le jury creuse : « pourquoi séparer ? » Réponse : c'est une partition verticale ; le stock change très souvent (à chaque vente) alors que la fiche de l'arbre est stable ; les séparer isole les écritures fréquentes et clarifie les responsabilités.

Migrations. Des scripts versionnés qui font évoluer le schéma de la base de façon reproductible et traçable. GreenRoots en compte vingt-quatre. Si le jury creuse : « à quoi ça sert par rapport à modifier la base à la main ? » Réponse : chaque changement est enregistré, rejouable à l'identique sur n'importe quel environnement (dev, CI, prod), et réversible dans l'historique Git ; c'est ce qui rend le schéma reproductible.

4.2 Accès aux données et transactions

ORM (Object-Relational Mapping). Une couche qui fait correspondre les tables de la base à des objets du code, pour manipuler les données sans écrire de SQL à la main. Prisma est mon ORM. Si le jury creuse : « avantage et inconvénient d'un ORM ? » Réponse :

avantage, la productivité et la sécurité (requêtes paramétrées, typage) ; inconvénient, on peut générer des requêtes non optimales sans s'en rendre compte, il faut savoir regarder le SQL produit.

Prisma. Mon ORM, en TypeScript, qui apporte un typage de bout en bout, des migrations et des requêtes paramétrées. Si le jury creuse : « Prisma génère quoi comme SQL pour un include imbriqué ? » Réponse : des jointures (JOIN) ; par exemple récupérer une commande avec ses articles, leurs arbres et le stock se traduit par des JOIN entre les tables commandes, articles, arbres et stock. Sache lire l'équivalence, c'est une attente explicite du jury.

Requête paramétrée. Une requête où les valeurs sont envoyées séparément de l'instruction SQL, via des marqueurs (comme \$1, \$2), et ne sont jamais interprétées comme du code. C'est la parade fondamentale contre l'injection SQL. Si le jury creuse : « la différence avec la concaténation ? » Réponse : concaténer une entrée utilisateur dans une chaîne SQL permet l'injection ; la paramétrer la traite comme une simple donnée, l'injection devient impossible.

Transaction et atomicité. Une transaction regroupe plusieurs opérations en un tout indivisible : soit tout réussit, soit tout est annulé (rollback), jamais un état intermédiaire. En SQL, cela s'écrit BEGIN au début et COMMIT à la fin (ou ROLLBACK en cas d'erreur) ; en Prisma, c'est la fonction transaction. Dans GreenRoots, la réservation de stock décrémente le stock et met à jour la commande dans une même transaction. Si le jury creuse : « que se passe-t-il si le serveur plante au milieu ? » Réponse : la transaction n'est pas validée, donc annulée automatiquement ; le stock n'est pas décrétement à moitié, l'intégrité est préservée.

ACID. Les quatre garanties d'une transaction : Atomicité (tout ou rien), Cohérence (on passe d'un état valide à un autre), Isolation (les transactions concurrentes ne se marchent pas dessus), Durabilité (une fois validée, c'est écrit durablement). Si le jury creuse : « expliquez l'isolation dans votre cas. » Réponse : deux clients qui achètent le dernier arbre en même temps ne peuvent pas décrétement le stock deux fois ; le décrétement atomique en base et la transaction garantissent qu'un seul passe, l'autre voit le stock insuffisant.

Idempotence. Une opération est idempotente si l'exécuter une ou plusieurs fois produit le même résultat. Crucial pour le paiement. Dans GreenRoots : si l'utilisateur double-clique sur « Payer », un drapeau (stockReservedAt, plus le statut PROCESSING) évite de réserver le stock deux fois ; et le webhook Stripe, s'il est rejoué, ne repasse pas une commande déjà payée. Si le jury creuse : « donnez un contre-exemple non idempotent. » Réponse : un simple « décrémente le stock de 1 » rejoué deux fois retirerait 2 ; c'est pourquoi je garde une marque d'état pour ne rejouer que si nécessaire.

Race condition (situation de compétition). Un bug qui survient quand deux opérations concurrentes accèdent à la même ressource sans coordination, produisant un résultat incohérent (par exemple sur-vendre un stock). Parade : l'opération atomique en base et la transaction. Si le jury creuse : « comment évitez-vous la sur-vente ? » Réponse : le

décrément se fait en une seule instruction SQL atomique (quantité égale quantité moins X), et la lecture-écriture est dans une transaction, donc deux commandes simultanées ne peuvent pas passer sous le stock réel.

4.3 NoSQL et Redis

NoSQL. Des bases qui ne suivent pas le modèle relationnel : clé-valeur, document, colonne, graphe. On les choisit pour la vitesse, la simplicité ou la mise à l'échelle sur certains usages. Redis est une base NoSQL clé-valeur. Si le jury creuse : « SQL ou NoSQL, comment choisir ? » Réponse : SQL pour des données structurées et des relations fortes avec intégrité (mes commandes) ; NoSQL clé-valeur pour du cache et des données éphémères à accès ultra-rapide (Redis) ; les deux cohabitent, chacun sur son usage.

Redis. Une base clé-valeur qui vit en mémoire vive, donc extrêmement rapide (de l'ordre de la fraction de milliseconde). Trois usages dans GreenRoots : le cache, l'anti-force-brute, et la liste noire des JWT. Si le jury creuse : « pourquoi en mémoire, ce n'est pas risqué ? » Réponse : la volatilité est acceptable ici parce que ces données sont soit restructurables (le cache se recharge depuis la base), soit temporaires (compteurs, jetons expirants) ; je n'y mets rien de critique et non restructurable.

Cache-aside (cache à côté). Un patron de cache : on lit d'abord le cache ; en cas d'absence (miss), on lit la base, puis on remplit le cache pour la prochaine fois. Dans GreenRoots, les pays sont mis en cache vingt-quatre heures, les catégories une heure avec invalidation à l'écriture. Si le jury creuse : « qu'est-ce que l'invalidation ? » Réponse : quand une donnée change en base (une catégorie modifiée), on efface son entrée de cache pour ne pas servir une valeur périmée ; le cache se reconstruit à la lecture suivante.

TTL (Time To Live, durée de vie). La durée après laquelle une entrée de cache expire automatiquement. Si le jury creuse : « comment choisit-on un TTL ? » Réponse : selon la fraîcheur nécessaire ; les pays changent quasi jamais, donc vingt-quatre heures ; les catégories peuvent bouger, donc une heure, complétée par l'invalidation immédiate à l'écriture.

Dégradation gracieuse et fail-open. Si Redis tombe, l'application continue de fonctionner en se rabattant sur la base de données ; le cache accélère mais n'est pas critique. Fail-open signifie qu'en cas de panne du garde-fou, on laisse passer plutôt que de tout bloquer. Si le jury creuse : « fail-open, n'est-ce pas un risque de sécurité ? » Réponse : c'est un arbitrage disponibilité contre confidentialité ; pour l'anti-force-brute, je préfère que le login reste disponible si Redis tombe plutôt que de bloquer tous les utilisateurs ; c'est un choix conscient, pas un oubli.

SPOF (Single Point Of Failure, point de défaillance unique). Un composant dont la panne fait tomber tout le système. Justement, grâce à la dégradation gracieuse, Redis n'est pas un SPOF chez moi. Si le jury creuse : « quels SPOF reste-t-il ? » Réponse : la base

PostgreSQL en est un (single instance) ; en production sérieuse on ajouterait de la réplication ; c'est un axe identifié.

Machine à états (state machine). Un modèle où un objet ne peut passer que par des transitions autorisées entre des états définis. Dans GreenRoots, le statut de commande est une vraie machine à états : une constante déclare les transitions permises (par exemple de PENDING vers CANCELLED, de CONFIRMED vers PLANTED), et un garde refuse toute transition non listée. Nuance à assumer : seul le statut de commande a cette table de transitions explicite ; le statut de paiement, lui, est un enum piloté par le webhook Stripe, sans table de transitions formelle. Si le jury creuse : « avez-vous deux machines à états ? » Réponse : une seule formelle, celle du statut de commande ; le paiement suit des transitions implicites dictées par Stripe. Ne survends pas.

5. Frontend

React. Une bibliothèque JavaScript pour construire des interfaces à base de composants réutilisables, avec un rendu déclaratif. Si le jury creuse : « déclaratif, ça veut dire quoi ? » Réponse : je décris à quoi l'interface doit ressembler pour un état donné, et React se charge de mettre à jour le DOM ; je ne manipule pas le DOM à la main.

Vite. Un outil de build et un serveur de développement très rapide pour les applications front modernes. Si le jury creuse : « pourquoi Vite et pas Webpack ? » Réponse : démarrage quasi instantané et rechargement à chaud rapide en développement, build optimisé en production ; c'est plus léger et moderne que Webpack, et ça sert l'éco-conception.

Les trois natures d'état. Un point fort à mettre en avant : je ne mets pas tout dans un état global, je sépare l'état selon sa nature. L'état local de l'interface (le panier) avec Zustand ; l'état serveur (le stock, source de vérité) avec TanStack Query ; l'état de formulaire avec React Hook Form et Zod. Si le jury creuse : « pourquoi cette séparation ? » Réponse : chaque type d'état a des besoins différents (persistance, synchronisation, validation) ; un seul store global mélangerait tout et deviendrait ingérable ; le bon outil pour chaque besoin.

Zustand. Une petite bibliothèque de gestion d'état global côté client. Le panier y est stocké, avec persistance dans le localStorage et une expiration de vingt-quatre heures. Nuance : ce n'est pas un TTL natif, c'est une expiration maison basée sur la date de création. Si le jury creuse : « pourquoi Zustand plutôt que Redux ? » Réponse : Redux est puissant mais verbeux ; Zustand offre l'essentiel avec beaucoup moins de code, adapté à la taille du projet.

TanStack Query. Une bibliothèque qui gère les données venant du serveur : cache, rafraîchissement, états de chargement et d'erreur. Le stock est rafraîchi par interrogation régulière (polling) toutes les cinq minutes et au retour sur l'onglet. Si le jury creuse : « c'est quoi staleTime ? » Réponse : la durée pendant laquelle une donnée est considérée comme fraîche et n'est pas refetchée ; au-delà, TanStack Query la rafraîchit ; les pays ont un staleTime d'une heure car ils changent peu.

syncWithStock (réconciliation panier - stock). Le panier est persisté localement, il peut donc devenir périmé ; à l'ouverture, on le réconcilie avec le stock serveur. Trois cas : produit introuvable (arbre supprimé côté serveur), l'article est conservé dans le panier mais masqué à l'affichage ; stock suffisant, inchangé ; stock insuffisant, la quantité est ajustée et une fenêtre prévient l'utilisateur. Si le jury creuse : « pourquoi ne pas juste supprimer l'article introuvable ? » Réponse : choix produit, on préfère le garder et informer plutôt que de vider silencieusement le panier de l'utilisateur.

Polling. Interroger le serveur à intervalle régulier pour récupérer des données à jour. On l'utilise pour le stock et pour attendre la confirmation de paiement. Si le jury creuse : « pourquoi pas du temps réel avec WebSocket ou SSE ? » Réponse : le polling est simple et suffisant pour ces besoins ; les Server-Sent Events seraient plus efficaces pour du temps réel, c'est un axe d'amélioration identifié dans les perspectives.

Responsive et useMediaQuery. Le responsive adapte l'interface à la taille de l'écran. Point fort : chez moi ce n'est pas que du CSS, le hook useMediaQuery fait carrément *changer de composant* - un tableau des commandes sur ordinateur devient des cartes sur mobile. Si le jury creuse : « différence avec du responsive CSS classique ? » Réponse : le CSS réarrange le même HTML ; ici, sous un certain seuil, je rends un composant différent, mieux adapté au tactile ; c'est un choix d'expérience utilisateur.

RGAA, WCAG, ARIA. Le RGAA est le référentiel français d'accessibilité, adossé au standard international WCAG. ARIA est un ensemble d'attributs HTML qui décrivent le rôle et l'état des éléments aux technologies d'assistance (lecteurs d'écran). Dans GreenRoots, les composants shadcn/ui apportent l'ARIA nativement, et mon champ de quantité expose un rôle spinbutton avec les valeurs min, max et courante. Si le jury creuse : « votre score d'accessibilité Lighthouse de 96 veut-il dire conforme RGAA ? » Réponse honnête : non ; Lighthouse teste automatiquement environ un tiers des critères ; une conformité RGAA complète demande un audit manuel ; 96 est un bon indicateur, pas une certification.

shadcn/ui et Radix. shadcn/ui est une collection de composants d'interface que l'on copie dans son projet, construits sur Radix, une bibliothèque de primitives accessibles et non stylées. Si le jury creuse : « pourquoi ce choix cochant toutes vos cases ? » Réponse : Radix apporte l'accessibilité, on importe seulement ce qu'on utilise (éco-conception), et c'est typé ; c'est le choix qui réunit mes trois fils directeurs.

Tailwind CSS. Un framework CSS utilitaire : on style directement dans le HTML avec des petites classes. Le build purge les classes inutilisées. Si le jury creuse : « ça ne rend pas le HTML illisible ? » Réponse : c'est le reproche courant ; en pratique, combiné à des composants, ça évite des feuilles de style énormes et ça garde le style au plus près du markup ; la purge réduit fortement le CSS final.

Type-safety et z.infer. La sûreté de typage garantit à la compilation qu'on manipule les bons types. Avec Zod, je définis un schéma de validation et j'en dérive automatiquement le type TypeScript avec z.infer, si bien que le schéma et le type ne peuvent jamais diverger. Si le

jury creuse : « quel intérêt concret ? » Réponse : une seule source de vérité pour les règles et le type ; si je modifie le schéma, le type suit, et le compilateur m'alerte partout où c'est incohérent.

6. Paiement (Stripe)

Stripe. Une plateforme de paiement en ligne. Point clé de sécurité : aucune donnée bancaire ne transite ni n'est stockée chez moi, c'est Stripe qui les gère ; je ne manipule qu'un identifiant de paiement. Si le jury creuse : « et la conformité PCI-DSS ? » Réponse : en déléguant la saisie de la carte à Stripe, je sors mon serveur du périmètre PCI-DSS le plus lourd ; c'est justement l'intérêt d'externaliser le paiement.

PaymentIntent. L'objet Stripe qui représente une tentative de paiement et suit son cycle de vie. Chaque commande a son PaymentIntent. Si le jury creuse : « pourquoi un PaymentIntent et pas un simple paiement ? » Réponse : il gère les étapes (authentification forte 3D Secure, échec, nouvelle tentative) et l'état du paiement, ce qui colle aux exigences européennes DSP2.

Le chemin du PaymentIntent, de bout en bout (front, back, Stripe). À savoir raconter d'une traite. Un, le front poste sur la route d'initialisation du checkout. Deux, le back, dans une transaction, crée le PaymentIntent chez Stripe en lui envoyant le montant, la devise et, dans les métadonnées, l'identifiant de la commande ; il stocke l'identifiant du PaymentIntent sur la commande et récupère le client_secret. Trois, le back renvoie au front l'identifiant de commande et le client_secret. Quatre, le front injecte ce client_secret dans les composants Stripe (la bibliothèque Elements) ; au clic sur payer, il appelle d'abord la confirmation côté back qui réserve le stock de façon atomique, puis il lance le paiement dans le navigateur avec le client_secret (la carte est saisie et envoyée directement à Stripe, jamais à mon serveur). Cinq, Stripe encaisse puis notifie mon back par webhook (événement paiement réussi) ; c'est le webhook, et lui seul, qui fait passer la commande en payée et confirmée, génère la facture et envoie l'email. Point clé à marteler : le résultat du paiement n'est jamais décidé par le navigateur, le front attend la confirmation du serveur (issue du webhook) par interrogation régulière. Les quatre secrets Stripe à ne pas confondre : la clé publiable (front, publique, pour initialiser Stripe.js), la clé secrète (back, pour appeler l'API Stripe), le secret de webhook (back, pour vérifier la signature HMAC des notifications), et le client_secret (propre à chaque PaymentIntent, envoyé au front pour confirmer ce paiement précis).

Webhook. Une notification que Stripe envoie à mon serveur quand un événement se produit (paiement réussi, échoué, annulé). C'est la source de vérité du paiement. Si le jury creuse : « pourquoi la source de vérité est le webhook et pas la réponse du navigateur ? »

Réponse : le navigateur est manipulable, on ne peut pas lui faire confiance pour valider un paiement ; le webhook vient directement de Stripe, serveur à serveur ; mon front attend d'ailleurs la confirmation serveur par polling.

Signature de webhook (constructEvent). Chaque webhook Stripe est signé en HMAC avec un secret partagé. Mon serveur vérifie cette signature avant tout traitement ; une signature invalide est rejetée avec une erreur 400. Il faut le corps brut de la requête (rawBody), non transformé, pour vérifier la signature. Si le jury creuse : « pourquoi le corps brut ? » Réponse : la signature est calculée sur les octets exacts envoyés ; si un middleware reformate le JSON, la signature ne correspond plus ; d'où la nécessité du corps brut.

Anti-spoofing (anti-usurpation). Vérifier qu'un webhook correspond bien à la bonne commande : je compare l'identifiant de paiement reçu à celui enregistré sur la commande. Si le jury creuse : « quel scénario ça bloque ? » Réponse : une tentative de faire confirmer une commande avec l'identifiant de paiement d'une autre ; s'ils ne correspondent pas, on ignore.

Cron de réconciliation. Une tâche planifiée (toutes les heures) qui rattrape les webhooks éventuellement perdus : elle réinterroge Stripe pour les commandes bloquées en cours de traitement et les finalise si le paiement a réussi. Si le jury creuse : « pourquoi en avoir besoin si le webhook marche ? » Réponse : un webhook peut se perdre (panne réseau, redémarrage) ; le cron est un filet de sécurité pour la disponibilité et l'intégrité, il évite qu'une commande payée reste bloquée. C'est le « D » de DICP.

Mode test Stripe et carte 4242. Stripe a un mode test qui simule des paiements sans argent réel, avec des numéros de carte de test comme 4242 4242 4242 4242. C'est ce que j'utilise pour la démo. Si le jury creuse : « la démo est un vrai paiement ? » Réponse : c'est un vrai flux technique de bout en bout, mais en mode test Stripe ; le mécanisme est identique en production, seule la clé change.

7. Tests

Test unitaire. Un test qui vérifie une petite unité de code isolée (une fonction, une méthode). GreenRoots en compte deux cent onze : cent cinquante et un côté back avec Jest, soixante côté front avec Vitest et Testing Library. Si le jury creuse : « pourquoi surtout de l'unitaire ? » Réponse : j'ai ciblé la logique critique (panier, checkout, paiement) où une régression coûte cher ; c'est un choix assumé, les tests d'intégration et de bout en bout sont dans mes perspectives.

Jest, Vitest, Testing Library. Jest est le lanceur de tests côté back ; Vitest son équivalent rapide côté front (aligné avec Vite) ; Testing Library teste les composants du point de vue de l'utilisateur (ce qu'il voit et fait), pas les détails internes. Si le jury creuse : « pourquoi tester du point de vue utilisateur ? » Réponse : ça rend les tests robustes au refactoring ; tant que le comportement visible ne change pas, le test passe, même si j'ai réécrit l'intérieur.

Arrange, Act, Assert. La structure d'un bon test : préparer le contexte, exécuter l'action, vérifier le résultat. Mon exemple : j'ajoute deux fois le même arbre au panier (arrange et act), puis je vérifie qu'il n'y a qu'une ligne avec la quantité cumulée (assert). Si le jury creuse : « c'est quoi un bon test ? » Réponse : rapide, isolé, déterministe, et qui teste un seul comportement clair.

Mock. Un faux objet qui remplace une dépendance réelle dans un test, pour isoler l'unité testée. Par exemple, un faux Prisma pour tester un service sans vraie base. Si le jury creuse : « mock ou vraie base ? » Réponse : mock en test unitaire pour la rapidité et l'isolation ; vraie base en test d'intégration (ma CI en lance une), pour vérifier les vraies requêtes.

Couverture (coverage). Le pourcentage de code exécuté par les tests. La CI la mesure sur la branche d'intégration. Si le jury creuse : « visez-vous 100 % ? » Réponse : non, viser 100 % est contre-productif ; je privilégie la couverture de la logique critique ; un seuil minimal en CI est un axe d'amélioration.

Jeu d'essai. Un ensemble de scénarios fonctionnels avec résultat attendu et résultat obtenu, pour valider un parcours de bout en bout. Mon jeu d'essai couvre le tunnel de vente : huit cas du nominal (ajout, paiement réussi) aux cas d'erreur (stock insuffisant, accès interdit, double confirmation, signature invalide). Si le jury creuse : « différence avec les tests unitaires ? » Réponse : le test unitaire vérifie une brique de code ; le jeu d'essai valide un comportement fonctionnel complet, tel que l'utilisateur ou le métier l'attend ; les deux sont demandés par le référentiel.

Tests d'intégration et de bout en bout (E2E). L'intégration teste plusieurs composants ensemble (par exemple l'API avec une vraie base) ; le bout en bout simule un utilisateur réel dans un navigateur, avec des outils comme Cypress ou Playwright. Si le jury creuse : « pourquoi ne pas les avoir faits ? » Réponse : arbitrage de temps sur un projet de formation ; je les ai identifiés et priorisés dans les perspectives, en connaissant leur valeur.

8. Déploiement et DevOps

Docker et conteneur. Docker empaquette une application et tout ce dont elle a besoin dans un conteneur : une unité isolée, légère et reproductible, qui tourne à l'identique partout. Si le jury creuse : « conteneur ou machine virtuelle ? » Réponse : une VM embarque un système d'exploitation complet, c'est lourd ; un conteneur partage le noyau de l'hôte et n'embarque que l'application, c'est bien plus léger et rapide à démarrer.

Image Docker. Le modèle figé à partir duquel on lance des conteneurs. Une image se construit à partir d'un Dockerfile. Si le jury creuse : « différence image et conteneur ? » Réponse : l'image est le gabarit (la classe), le conteneur est une instance en cours d'exécution (l'objet) ; d'une image on lance plusieurs conteneurs.

Docker Compose. Un outil pour décrire et lancer plusieurs conteneurs ensemble via un fichier. GreenRoots a un environnement de développement à sept services (front, back, PostGIS, Redis, Nginx, l'outil Stripe en ligne de commande, Prisma Studio) et un environnement de production durci à quatre services. Si le jury creuse : « pourquoi moins de services en production ? » Réponse : les outils de développement (Prisma Studio, le tunnel Stripe) et le hot reload n'ont rien à faire en production ; on réduit la surface d'attaque et le poids.

Multi-stage build, Alpine, non-root. Le multi-stage sépare la construction de l'image finale : on compile dans une première étape, et l'image finale ne garde que le nécessaire, donc légère. Alpine est une distribution Linux minimale. Non-root signifie que le conteneur ne tourne pas en administrateur. Si le jury creuse : « pourquoi non-root ? » Réponse : si un attaquant compromet le conteneur, il n'a pas les pleins pouvoirs ; c'est le principe du moindre privilège appliqué aux conteneurs.

Healthcheck. Une sonde qui vérifie régulièrement que le service répond, via une route de santé (chez moi /api/health). Si le jury creuse : « à quoi ça sert ? » Réponse : l'orchestrateur sait si le conteneur est réellement opérationnel, pas juste démarré ; il peut le redémarrer s'il ne répond plus, ce qui sert la disponibilité.

Nginx gzip et cache. En production, Nginx compresse les fichiers (gzip) pour réduire leur poids et met en cache les fichiers statiques longtemps (un an), avec un nom de fichier qui change à chaque build. Si le jury creuse : « comment invalidez-vous un cache d'un an ? » Réponse : par le hash dans le nom de fichier ; à chaque build le nom change, donc le navigateur télécharge la nouvelle version tout en gardant l'ancienne en cache tant qu'elle ne change pas.

Joi (validation des variables d'environnement). Au démarrage, Joi valide que toutes les variables d'environnement nécessaires sont présentes et correctes (par exemple, le secret JWT doit faire au moins trente-deux caractères), sinon l'application refuse de démarrer. Si le jury creuse : « pourquoi valider au démarrage ? » Réponse : principe du fail-fast ; mieux vaut planter tout de suite avec un message clair qu'au milieu d'une requête en production à cause d'une variable oubliée.

CI/CD (intégration et déploiement continu). L'intégration continue teste automatiquement chaque changement ; le déploiement continu automatise la mise en production. GreenRoots a trois workflows GitHub Actions calqués sur le git flow. Si le jury creuse : « différence CI et CD ? » Réponse : la CI, c'est vérifier automatiquement que le code s'intègre sans casse (lint, tests) ; le CD, c'est livrer automatiquement en production une fois la CI verte.

GitHub Actions, workflow, job. GitHub Actions est le moteur d'automatisation intégré à GitHub. Un workflow est un pipeline déclenché par un événement (une pull request, un push) ; il contient des jobs (étapes). Mes trois workflows : sur chaque pull request, lint plus tests plus audit de sécurité, c'est le garde-fou qui bloque la fusion ; sur la branche

d'intégration, une suite étendue avec une vraie base PostgreSQL, migrations, seed et couverture ; sur la branche principale, les tests puis le déclenchement du déploiement et la vérification de santé. Si le jury creuse : « que se passe-t-il si un test échoue en pull request ? » Réponse : la fusion est bloquée ; c'est le garde-fou qualité.

Lint, ESLint, Prettier. Le lint analyse le code pour détecter erreurs et mauvaises pratiques ; ESLint est le linter JavaScript/TypeScript, Prettier formate automatiquement le code. Si le jury creuse : « lint et formatage, c'est pareil ? » Réponse : non ; ESLint traque les problèmes de qualité et de bugs potentiels ; Prettier ne s'occupe que de la mise en forme (indentation, guillemets) ; les deux tournent en CI.

npm audit. Une commande qui vérifie si les dépendances du projet ont des vulnérabilités connues. En CI, je bloque sur les failles critiques (niveau critical). Si le jury creuse : « pourquoi seulement les critiques ? » Réponse : choix assumé pour ne pas bloquer sur du bruit de faible gravité ; les vulnérabilités critiques, elles, arrêtent la chaîne ; on interprète le rapport et on traite au cas par cas.

Coolify, PaaS, VPS, Hetzner. Un VPS est un serveur privé virtuel loué (le mien est chez Hetzner). Coolify est une plateforme (un PaaS auto-hébergé, une sorte de Heroku maison) qui orchestre Docker et gère les déploiements. Point fort à valoriser : j'ai réalisé ce déploiement de bout en bout moi-même. Si le jury creuse : « pourquoi auto-héberger plutôt qu'un service managé ? » Réponse : maîtrise complète de la chaîne et coût, dans une démarche d'apprentissage ; en contrepartie j'assume l'exploitation, d'où le monitoring en perspective.

HTTPS et sslip.io. HTTPS chiffre les échanges entre le navigateur et le serveur. sslip.io est un service qui fournit un nom de domaine à partir d'une adresse IP, pratique pour obtenir un certificat HTTPS sans acheter de domaine. Si le jury creuse : « pourquoi sslip.io ? » Réponse : c'est un projet de formation sans nom de domaine acheté ; sslip.io permet un HTTPS valide rapidement ; en vrai projet on prendrait un domaine dédié.

pg_dump et pg_restore (sauvegarde et restauration). pg_dump exporte une base PostgreSQL dans un fichier ; pg_restore la réimporte. J'ai une procédure documentée, validée en local. Nuance honnête : l'automatisation planifiée de ces sauvegardes reste à finaliser. Si le jury creuse : « votre sauvegarde est-elle automatisée en production ? » Réponse : la procédure est écrite et testée en local ; l'automatisation par tâche planifiée est le prochain pas, je l'assume dans les perspectives.

Makefile. Un fichier qui regroupe des commandes sous des noms courts (par exemple « make dev » pour tout lancer). GreenRoots en a une vingtaine. Si le jury creuse : « pourquoi un Makefile ? » Réponse : simplifier et documenter les commandes courantes ; un nouvel arrivant lance le projet en une commande, c'est de l'onboarding et de la reproductibilité.

9. RGPD et cadre légal

RGPD. Le Règlement Général sur la Protection des Données, le cadre européen sur les données personnelles. Il donne des droits aux personnes et des obligations aux traitements. Si le jury creuse : « quels droits avez-vous implémentés ? » Réponse : le droit d'accès (export des données), le droit à l'effacement (suppression de compte par anonymisation), et la minimisation (on ne renvoie jamais les champs sensibles).

Droit d'accès (article 15). L'utilisateur peut obtenir une copie de ses données. Chez moi, une route d'export renvoie un fichier JSON structuré : profil, commandes, adresses (pas le panier). Si le jury creuse : « accès ou portabilité ? » Réponse : la route s'appelle export au titre de l'accès (article 15) ; comme le format JSON est structuré et réutilisable, une lecture au titre de la portabilité (article 20) s'appliquerait aussi.

Droit à l'effacement (article 17) et anonymisation. L'utilisateur peut demander la suppression de ses données. Mais on ne peut pas tout supprimer, car les commandes servent de preuve comptable. La solution : anonymiser les données personnelles (le nom devient « Utilisateur supprimé », l'email un email neutre, l'adresse est effacée) puis supprimer le compte, le tout dans une transaction atomique ; les commandes, elles, sont conservées mais rattachées à un utilisateur anonyme. Si le jury creuse : « anonymisation ou pseudonymisation ? » Réponse : anonymisation, car le lien vers la personne réelle est irréversiblement rompu ; la pseudonymisation garderait une table de correspondance permettant de ré-identifier, ce n'est pas mon cas.

Obligation de conservation. Certaines données (factures, commandes) doivent être conservées un certain temps pour des raisons comptables et fiscales, généralement six ans. C'est ce qui justifie l'anonymisation plutôt que la suppression pure. Piège juridique à éviter : je dis « obligation légale de conservation, six ans » sans citer un article précis qui pourrait être faux ; ne cite pas un numéro d'article dont tu n'es pas sûr. Si le jury creuse : « le droit à l'effacement ne prime-t-il pas ? » Réponse : non, le RGPD prévoit des exceptions quand une obligation légale impose la conservation ; on concilie les deux en anonymisant les données personnelles tout en gardant les documents comptables.

Minimisation et safeSelect. Le principe de minimisation dit de ne traiter que les données nécessaires. Chez moi, une fonction safeSelect exclut par défaut le mot de passe et les jetons hachés de toutes les réponses de l'API. Si le jury creuse : « c'est de la minimisation ou de la sécurité ? » Réponse : les deux se rejoignent ; ne jamais renvoyer un champ sensible protège la confidentialité et respecte la minimisation en sortie.

Cookies et ePrivacy. La directive ePrivacy impose une bannière de consentement pour les cookies non essentiels (traceurs, analytics). Point fort : GreenRoots n'a aucun traceur ni analytics, le seul cookie est celui d'authentification, strictement nécessaire, donc exempté de

bannière. Si le jury creuse : « pourquoi pas de bannière cookies ? » Réponse : parce que le seul cookie est fonctionnel et strictement nécessaire (l'authentification) ; la loi n'exige de consentement que pour les cookies non essentiels, que je n'ai pas.

Facture et numérotation séquentielle. Chaque paiement réussi génère une facture avec un numéro séquentiel unique, au format FA, année, numéro sur six chiffres, généré dans une transaction pour garantir l'unicité et l'absence de trou. Si le jury creuse : « pourquoi une numérotation séquentielle sans trou ? » Réponse : c'est une exigence légale de facturation ; la générer en transaction évite les doublons et les sauts, même en cas d'accès concurrents ; c'est le « P » de DICP, la preuve.

10. Services externes et divers

Clouinary. Un service de gestion et d'optimisation d'images dans le cloud. Les images des arbres y sont hébergées et servies optimisées. Si le jury creuse : « pourquoi externaliser les images ? » Réponse : optimisation automatique (formats, tailles, compression), distribution rapide, et on n'encombre pas notre serveur ; ça sert la performance et l'éco-conception.

Resend. Un service d'envoi d'emails transactionnels (confirmation de commande, réinitialisation de mot de passe). Si le jury creuse : « pourquoi un service tiers pour les emails ? » Réponse : la délivrabilité ; envoyer des emails depuis son propre serveur finit souvent en spam ; un service spécialisé gère la réputation d'envoi.

Cron (tâche planifiée). Une tâche qui s'exécute automatiquement à intervalle régulier. Chez moi, le cron de réconciliation des paiements tourne toutes les heures. Si le jury creuse : « où tourne ce cron ? » Réponse : dans le back NestJS, via son planificateur intégré ; il n'a pas besoin d'un service externe.

Swagger / OpenAPI. Une spécification standard pour documenter une API REST, avec une interface web pour explorer et tester les routes. Si le jury creuse : « votre API est-elle documentée ? » Réponse : oui, via OpenAPI/Swagger, ce qui sert de contrat entre le front et le back et facilite les tests manuels.

Comment écouter ce glossaire en podcast sur ton téléphone

Ces deux fichiers sont du texte : n'importe quel outil de synthèse vocale (text-to-speech) peut les lire à voix haute. Le plus simple sur ton Pixel, avec de vraies voix naturelles :

Option 1 - Microsoft Edge (gratuit, voix françaises très naturelles, recommandé). 1. Installe l'application Edge sur le Pixel. 2. Transfère-toi le fichier en PDF ou ouvre-le : le plus simple est que je te génère une version PDF (demande-la-moi), tu

l'ouvres dans Edge, ou tu colles le texte dans une page. 3. Menu (les trois points) puis « Lecture à voix haute ». Tu choisis la voix (prends une voix française « Naturel »), la vitesse, et ça se met en lecture, y compris écran éteint.

Option 2 - @Voice Aloud Reader (application Android dédiée). 1. Installe « @Voice Aloud Reader » depuis le Play Store. 2. Importe le fichier texte ou PDF (l'appli lit TXT, PDF, HTML, ePub). 3. Elle lit avec la synthèse vocale du téléphone, en tâche de fond, avec une file de lecture, comme un vrai podcast.

Option 3 - Google Play Livres. 1. Envoie-toi le fichier en PDF ou ePub (via Google Drive ou email). 2. Importe-le dans Play Livres (« Importer des fichiers »). 3. Ouvre le livre, menu, « Lire à voix haute » (lecture automatique). Pratique pour la lecture en fond.

Option 4 - Speechify ou NaturalReader. Applications spécialisées, voix très naturelles, gratuites avec limites. Tu colles le texte ou importes le fichier, et tu écoutes comme un podcast.

Conseil de format. Les symboles Markdown (les dièses des titres, les tirets) peuvent être lus bizarrement par certaines voix. Si tu veux une écoute vraiment fluide, je peux te générer une **version « script audio »** : le même contenu en texte continu, sans symboles, découpé en chapitres, prêt à être lu d'une traite. Dis-le-moi et je te la prépare (et je peux aussi te sortir un PDF propre des deux parties).